Control Barrier Functions (CBF) — A Beginner-Friendly Explanation

> **Viewing:** Open in VS Code and press **Cmd+Shift+V** (Mac) or **Ctrl+Shift+V** (Windows) to open the preview with rendered equations.

The Big Picture

Imagine you have a robot that needs to travel from point A to point B while avoiding obstacles. You already have a controller that drives the robot toward the goal (say, a simple PD controller). The problem? That controller knows nothing about obstacles — it will happily steer the robot straight into a wall.

Control Barrier Functions (CBF) solve this by acting as a **safety filter**. You design your favourite goal-seeking controller first, then the CBF modifies it *just enough* to guarantee safety — and no more. It's like having a co-pilot who only grabs the steering wheel when you're about to crash, and otherwise lets you drive however you want.

The mathematical tool behind CBF is a **Quadratic Program (QP)** — a small optimisation problem solved at every timestep:

> "Find the control input u that is as close as possible to what my nominal controller wants, **subject to** the constraint that the robot stays safe."

This makes CBF fundamentally different from both APF and DAF. APF and DAF *bake* the obstacle avoidance directly into the controller formula. CBF keeps the nominal controller separate and only intervenes when needed.

Key differences at a glance

Feature	**APF (Classical)**	**DAF (Dissipative)**	**CBF (Barrier)**
Avoidance mechanism	Repulsive *force* (pushes away)	Dissipative *braking* (slows down)	Safety *constraint* (filters control)
Depends on	Position only	Position **and** velocity	Position **and** velocity
How it works	Add repulsive force to controller	Add directional braking to controller	Solve QP: minimally modify any controller

Feature	APF (Classical)	DAF (Dissipative)	CBF (Barrier)
Nominal controller	Tightly coupled (built-in)	Tightly coupled (built-in)	**Decoupled** — any controller works
Local minima	Prone to getting stuck	Less likely (but flat walls)	Possible, depends on nominal controller
Computational cost	Cheap (closed-form)	Cheap (closed-form)	Moderate (QP solve per timestep)
Safety guarantee	No formal proof	Proven (Lyapunov + barrier)	Proven (forward invariance)
Extensibility	Hard to add constraints	Hard to add constraints	Easy — add more QP constraints

---

The Paper's Logic — Before the Equations

The standard CBF development follows this narrative thread:

1. **Model the robot** (Eq. 1) — state that the system is control-affine (a common and general robot model).
2. **Define safety** (Eqs. 2-3) — formalise what "safe" means as a set defined by a function $h(x) \geq 0$.
3. **Define the barrier function** (Eqs. 4-5) — introduce class $\mathcal{K}$ functions and the CBF condition that enforces safety.
4. **Handle second-order dynamics** (Eqs. 6-9) — extend CBF to systems where control affects acceleration, not velocity (relative degree 2).
5. **Design the nominal controller** (Eq. 10) — build a goal-seeking controller independently of safety.
6. **Formulate the QP** (Eq. 11) — combine the nominal controller with the safety constraint into an optimisation problem.
7. **Prove safety** (Eqs. 12-13) — show that the safe set is forward invariant (once safe, always safe).
8. **Handle multiple obstacles** (Eq. 14) — extend to multiple simultaneous safety constraints.
9. **Apply to obstacle avoidance** (Eqs. 15-17) — specialise the general theory to a robot avoiding obstacles.
10. **Compare to DAF and APF** (Eqs. 18-19) — benchmark against the other methods.

Master Symbol Table

Every symbol used in the CBF framework, grouped by category.

Spaces and Sets

Symbol	Type	Meaning
$\mathbb{R}$	set	Real numbers
$\mathbb{R}^n$	set	n-dimensional Euclidean space
$\mathbb{R}^m$	set	m-dimensional control input space
n	$\mathbb{N}$	Dimension of the state space
m	$\mathbb{N}$	Dimension of the control input
$\mathcal{C}$	$\subset \mathbb{R}^n$	**Safe set** — the set of states where $h(x) \geq 0$
$\partial\mathcal{C}$	set	**Boundary** of the safe set — where $h(x) = 0$
$\text{int}(\mathcal{C})$	set	**Interior** of the safe set — where $h(x) > 0$
$\mathcal{U}$	$\subseteq \mathbb{R}^m$	Set of admissible control inputs

State Variables

Symbol	Type	Meaning
x	$\in \mathbb{R}^n$	**Full state** of the system (e.g., $x = [p^\top, v^\top]^\top$)
p	$\in \mathbb{R}^{n_p}$	**Position** of the robot's centre
v	$\in \mathbb{R}^{n_p}$	**Velocity** of the robot ($\dot{p} = v$)
u	$\in \mathbb{R}^m$	**Control input** — what we command (acceleration for a double integrator)
$\dot{x}$	$\in \mathbb{R}^n$	Time derivative of the state
p_d	$\in \mathbb{R}^{n_p}$	**Desired target position** (the goal)
R	$\in \mathbb{R}_{>0}$	**Radius** of the robot

System Dynamics

Symbol	Type	Meaning
$f(x)$	$\mathbb{R}^n \to \mathbb{R}^n$	**Drift dynamics** — how the state evolves with no control input
$g(x)$	$\mathbb{R}^n \to \mathbb{R}^{n \times m}$	**Control matrix** — how the control input affects the state
$\dot{x} = f(x) + g(x)u$	ODE	The **control-affine** system model

Barrier Function Variables

Symbol	Type	Meaning
$h(x)$	$\mathbb{R}^n \to \mathbb{R}$	**Barrier function** — positive means safe, zero means boundary, negative means unsafe
$\dot{h}(x, u)$	$\mathbb{R}$	Time derivative of h along the system trajectory
$L_f h(x)$	$\mathbb{R}$	**Lie derivative** of h along f: $\nabla h(x)^\top f(x)$ — how h changes due to drift
$L_g h(x)$	$\mathbb{R}^{1 \times m}$	**Lie derivative** of h along g: $\nabla h(x)^\top g(x)$ — how control affects h
$\nabla h(x)$	$\mathbb{R}^n$	**Gradient** of h — points in the direction of increasing safety
$\alpha(\cdot)$	function	**Extended class $\mathcal{K}_\infty$ function** — controls how aggressively safety is enforced

Design Parameters

Symbol	Type	Meaning	Typical Value
k_1	$\mathbb{R}_{>0}$	Position gain for the nominal PD controller	1-5
k_2	$\mathbb{R}_{>0}$	Velocity damping gain for nominal controller	2-4
α_0	$\mathbb{R}_{>0}$	CBF enforcement strength (when $\alpha(h) = \alpha_0 h$)	1-10
α_1, α_2	$\mathbb{R}_{>0}$	ECBF pole placement parameters for second-order systems	1-5
ϵ	$\mathbb{R}_{>0}$	Safety margin beyond robot radius	0.06

Obstacle Variables

Symbol	Type	Meaning
p_o	$\in \mathbb{R}^{n_p}$	Position of obstacle centre

Symbol	Type	Meaning
r_o	$\in \mathbb{R}_{>0}$	Radius of obstacle (for circular obstacles)
$d(p)$	$\mathbb{R}$	Distance from robot surface to obstacle surface
$\eta(p)$	$\in \mathbb{R}^{n_p}$, $\lvert\eta\rvert = 1$	Unit vector pointing from obstacle toward robot
N_{obs}	$\mathbb{N}$	Number of obstacles

QP Variables

Symbol	Type	Meaning
u_{nom}	$\in \mathbb{R}^m$	**Nominal control** — what the goal-seeking controller wants
u^*	$\in \mathbb{R}^m$	**Optimal control** — output of the QP (safe version of u_{nom})
$\lvert u - u_{\text{nom}}\rvert^2$	$\mathbb{R}_{\geq 0}$	**Cost function** — deviation from nominal (minimised by QP)

Every Equation Explained

Equation (1) — Control-affine system dynamics

$$\dot{x} = f(x) + g(x)\, u$$

Why it's here: Before defining safety, we need to say what kind of system we're controlling. CBF theory requires the system to be **control-affine** — the control input u enters the dynamics linearly. This is a broad class that includes most robots.

What it is: The general model of a nonlinear system where the state x evolves under drift $f(x)$ (what happens with no input) plus the effect of the control $g(x)u$.

Intuition: Think of $f(x)$ as gravity or friction — forces that act regardless of what you do. And $g(x)u$ as your thrusters or motors — forces you choose. The key requirement is that u enters **linearly** (no u^2 or $\sin(u)$ terms). This is what makes the QP in Eq. 11 tractable.

Variable drill-down:

Variable	Dimension	What it is here
$x \in \mathbb{R}^n$	n	Full state vector. For a robot: $x = [p^\top, v^\top]^\top$ (position and velocity stacked).
$\dot{x} \in \mathbb{R}^n$	n	Time derivative of the state — how it changes.
$f(x)$	$n \times 1$	**Drift** — the dynamics when $u = 0$. For a double integrator: $f(x) = [v^\top, 0^\top]^\top$ (position changes at rate v, velocity doesn't change on its own).
$g(x)$	$n \times m$	**Input matrix** — maps control to state change. For a double integrator: $g(x) = [0; I]$ (control affects velocity, not position directly).
$u \in \mathbb{R}^m$	m	Control input. For a robot in 2D: $u = [u_x, u_y]^\top$ (acceleration command).

For a double integrator robot (the same model as DAF's Eq. 5):

$$x = \begin{bmatrix} p \\ v \end{bmatrix}, \quad f(x) = \begin{bmatrix} v \\ 0 \end{bmatrix}, \quad g(x) = \begin{bmatrix} 0 \\ I \end{bmatrix}$$

This gives $\dot{p} = v$ and $\dot{v} = u$, identical to DAF.

Equation (2) — The safe set

$$\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}$$

Why it's here: Safety needs a precise mathematical definition. Instead of vaguely saying "don't hit obstacles," we define a specific region of state space where the robot is safe. The entire CBF machinery exists to keep $x(t)$ inside this set for all time.

What it is: The safe set — the collection of all states where the barrier function h is non-negative.

Intuition: Draw a line around all the "good" states. Everything inside (where $h > 0$) is safe. The boundary ($h = 0$) is the edge of safety. Outside ($h < 0$) is collision. The CBF's job is to ensure the system never crosses from inside to outside.

Variable drill-down:

Variable	What it is here
$\mathcal{C}$	The **safe set** — all states that are acceptable
x	Any candidate state
$h(x)$	The barrier function evaluated at x — a single number that tells you "how safe" you are
$h(x) \geq 0$	The safety condition — non-negative means safe

Variable	What it is here
$h(x) = 0$	The boundary $\partial\mathcal{C}$ — the robot is at the edge of safety
$h(x) > 0$	The interior $\text{int}(\mathcal{C})$ — the robot is strictly safe
$h(x) < 0$	**Unsafe** — collision has occurred or is imminent

Key insight: The choice of h is a design decision. Different choices of h define different safe sets. For obstacle avoidance, a natural choice is $h(p) = \|p - p_o\|^2 - (R + r_o)^2$ (positive when the robot is far enough from the obstacle).

Equation (3) — The barrier function for obstacle avoidance

$$h(p) = \|p - p_o\|^2 - (R + r_o + \epsilon)^2$$

Why it's here: Equation (2) defined safety abstractly. Now we specialise to obstacle avoidance by choosing a concrete h. This particular choice encodes "the robot's centre must be at least $R + r_o + \epsilon$ away from the obstacle centre."

What it is: A barrier function for a circular robot avoiding a circular obstacle, with a safety margin.

Intuition: Picture two circles — the robot (radius R) and the obstacle (radius r_o). They collide when their centres are closer than $R + r_o$. We add a margin ϵ for extra safety. So:

- $h > 0$: the circles are separated (safe)
- $h = 0$: the circles are just touching (boundary)
- $h < 0$: the circles overlap (collision)

We use the **squared** distance $\|p - p_o\|^2$ instead of $\|p - p_o\|$ because: (a) it avoids a square root, making derivatives cleaner, and (b) the gradient ∇h is well-defined everywhere (no $0/0$ issue at $p = p_o$).

Variable drill-down:

Variable	What it is here
$h(p)$	Barrier function — positive means no collision
$p \in \mathbb{R}^{n_p}$	Robot centre position
$p_o \in \mathbb{R}^{n_p}$	Obstacle centre position
$\|p - p_o\|^2$	Squared Euclidean distance between centres: $(p_x - p_{o,x})^2 + (p_y - p_{o,y})^2$
R	Robot radius

Variable	What it is here
r_o	Obstacle radius
ϵ	Safety margin (same role as in DAF)
$(R + r_o + \epsilon)^2$	Squared minimum allowed distance between centres

For a flat wall at position $x = x_w$ (to compare with the DAF wall simulation):

$$h(p) = (p_x - x_w)^2 - (R + \epsilon)^2$$

or equivalently using signed distance:

$$h(p) = d(p) \cdot (d(p) + 2(R + \epsilon))$$

where $d(p) = |p_x - x_w| - (R + \epsilon)$ is the effective distance (same as DAF's $d(p)$).

Equation (4) — Extended class $\mathcal{K}_\infty$ function

$$\alpha : \mathbb{R} \to \mathbb{R}, \quad \text{strictly increasing}, \quad \alpha(0) = 0$$

Why it's here: The CBF condition (Eq. 5) uses a function α to control *how aggressively* safety is enforced. A steep α means the CBF intervenes strongly when h is small; a gentle α means it waits until the last moment. The class $\mathcal{K}_\infty$ requirement ensures the function behaves sensibly.

What it is: A function that passes through the origin, is strictly increasing, and is unbounded in both directions.

Intuition: Think of α as the CBF's "personality":

- $\alpha(h) = \alpha_0 \cdot h$ (linear): the most common choice. Aggressive enforcement — proportional to how far from the boundary you are. Like a spring that pulls you back harder the closer you get.
- $\alpha(h) = \alpha_0 \cdot h^3$ (cubic): gentler near the boundary, more aggressive far away.
- Large α_0: very aggressive — the CBF allows the robot to get very close to obstacles because it's confident it can stop in time.
- Small α_0: conservative — keeps a larger buffer.

Variable drill-down:

Variable	What it is here
$\alpha(\cdot)$	The class $\mathcal{K}_\infty$ function — a design choice
$\alpha(0) = 0$	When $h = 0$ (at the boundary), the right-hand side of the CBF condition is zero — the constraint is tightest

Variable	What it is here
$\alpha(h) > 0$ when $h > 0$	When safe, the constraint allows some decrease in h
$\alpha(h) < 0$ when $h < 0$	When unsafe, the constraint forces h to increase (recover)
"strictly increasing"	More safety margin (h larger) = more freedom to decrease h

Common choices:

$\alpha(h)$	Name	Behavior
$\alpha_0 \cdot h$	Linear	Proportional enforcement, most popular
$\alpha_0 \cdot h^3$	Cubic	Gentler near boundary
$\alpha_0 \cdot \tanh(h)$	Saturating	Caps the allowed decrease rate

---

Equation (5) — The CBF condition (first-order)

$$L_f h(x) + L_g h(x)\, u + \alpha(h(x)) \geq 0$$

Why it's here: This is the **core** of CBF theory — the single inequality that guarantees safety. If the control input u satisfies this condition at every instant, then $h(x(t)) \geq 0$ for all future time (the safe set is **forward invariant**). Everything else in the CBF framework exists to support or extend this one condition.

What it is: A constraint on the control input u that ensures h doesn't decrease too fast.

Intuition — step by step:

The time derivative of h along the system trajectory is:

$$\dot{h}(x, u) = \nabla h(x)^\top \dot{x} = \nabla h(x)^\top [f(x) + g(x)u] = L_f h(x) + L_g h(x)\, u$$

The CBF condition says:

$$\dot{h} \geq -\alpha(h)$$

In words: "h is allowed to decrease, but not faster than $\alpha(h)$." Since $\alpha(h) > 0$ when $h > 0$, the allowed rate of decrease shrinks as h approaches zero. At $h = 0$ (the boundary), $\alpha(0) = 0$, so $\dot{h} \geq 0$ — the system can't cross the boundary.

Analogy: Imagine driving toward a cliff. The CBF condition says: "You can accelerate toward the cliff, but only slowly enough that you could still brake in time." The closer you get, the tighter the speed limit, until at the edge you're required to be moving away or stopped.

Variable drill-down:

Variable	Type	What it is here
$L_f h(x)$	Scalar	**Lie derivative along** f: $\nabla h(x)^\top f(x)$. How h changes due to drift alone (no control).
$L_g h(x)$	$1 \times m$ row vector	**Lie derivative along** g: $\nabla h(x)^\top g(x)$. How control affects h.
$L_g h(x) \cdot u$	Scalar	The dot product — how much the chosen control u contributes to $\dot{h}$.
$\alpha(h(x))$	Scalar	The "allowed decrease rate" — bigger means more freedom.
$\nabla h(x)$	$n \times 1$	Gradient of h — direction of steepest increase in safety.
≥ 0	Constraint	The whole left side must be non-negative: $\dot{h} + \alpha(h) \geq 0$, i.e., $\dot{h} \geq -\alpha(h)$.

Why this guarantees safety: If $\dot{h} \geq -\alpha(h)$ always, then by comparison with the ODE $\dot{z} = -\alpha(z)$ (which has the solution $z(t) \geq 0$ for $z(0) \geq 0$ since $\alpha(0) = 0$), we get $h(x(t)) \geq 0$ for all t. This is Nagumo's theorem applied to the safe set.

Equation (6) — The relative degree problem

$$L_g h(x) = 0 \quad \forall x \in \mathcal{C}$$

Why it's here: For a double integrator robot ($\dot{p} = v$, $\dot{v} = u$), the barrier function $h(p)$ depends only on position, but control u affects acceleration. This means u doesn't appear in $\dot{h}$ — you can't directly constrain u using Eq. 5. The system has **relative degree 2**: you need to differentiate h twice before u appears.

What it is: The condition that signals the first-order CBF approach (Eq. 5) won't work directly.

Intuition: If h depends only on p and u affects v, then:

$$\dot{h} = \nabla_p h \cdot \dot{p} = \nabla_p h \cdot v$$

No u anywhere! The control input has no *direct* effect on $\dot{h}$. It's like trying to stop a car by controlling the engine: the engine controls acceleration, acceleration controls velocity, and velocity controls

position. You need two steps, not one.

Variable drill-down:

Variable	What it is here
$L_g h(x) = \nabla h^\top g(x)$	For $h = h(p)$ and $g = [0; I]$: $\nabla h = [\nabla_p h; 0]$, so $\nabla h^\top g = 0^\top I = 0$
"Relative degree 2"	u appears in $\ddot{h}$ but not in $\dot{h}$ — need to differentiate twice

This motivates the Exponential CBF (ECBF) approach in the next equations.

Equation (7) — Second derivative of the barrier function

$$\ddot{h} = \frac{\partial \dot{h}}{\partial x}(f(x) + g(x)u) = L_f^2 h(x) + L_g L_f h(x)\, u$$

Why it's here: Since u doesn't appear in $\dot{h}$ (Eq. 6), we differentiate again. Now u appears in $\ddot{h}$, so we can constrain it.

What it is: The second time derivative of h along the system trajectory, expanded using Lie derivatives.

Intuition: Continuing the car analogy: h is distance to cliff, $\dot{h}$ is how fast the distance changes (depends on velocity), $\ddot{h}$ is how fast $\dot{h}$ changes (depends on acceleration = control). Now we can say: "choose acceleration u so that $\ddot{h}$ satisfies some condition."

Variable drill-down:

Variable	Type	What it is here
$\ddot{h}$	Scalar	Second time derivative of h — how the rate-of-change of safety is changing
$L_f^2 h(x)$	Scalar	Lie derivative of $L_f h$ along f. The part of $\ddot{h}$ that doesn't depend on u.
$L_g L_f h(x)$	$1 \times m$ row vector	Lie derivative of $L_f h$ along g. How u affects $\ddot{h}$. For a double integrator with $h = \lvert p - p_o \rvert^2 - r^2$: this equals $2(p - p_o)^\top$.

For the obstacle avoidance barrier $h = \|p - p_o\|^2 - r_{\text{safe}}^2$:

$$\dot{h} = 2(p - p_o)^\top v$$

$$\ddot{h} = 2\|v\|^2 + 2(p - p_o)^\top u$$

Now u appears explicitly. $L_f^2 h = 2\|v\|^2$ and $L_g L_f h = 2(p - p_o)^\top$.

Equation (8) — Exponential CBF (ECBF) condition

$$\ddot{h} + \alpha_1 \dot{h} + \alpha_2 h \geq 0$$

equivalently:

$$L_f^2 h + L_g L_f h \cdot u + \alpha_1 (L_f h) + \alpha_2 h \geq 0$$

Why it's here: We need a condition on u (which appears in $\ddot{h}$) that keeps $h \geq 0$. The ECBF approach treats h like a damped spring: the condition $\ddot{h} + \alpha_1 \dot{h} + \alpha_2 h \geq 0$ ensures h behaves at least as well as the solution to $\ddot{z} + \alpha_1 \dot{z} + \alpha_2 z = 0$, which stays non-negative if the "poles" are chosen correctly.

What it is: The second-order CBF constraint — a linear inequality in u that guarantees safety for systems with relative degree 2.

Intuition: The condition shapes how h can evolve over time:

- $\alpha_2 h$ term: when h is large (far from obstacle), allows more freedom
- $\alpha_1 \dot{h}$ term: when $\dot{h} < 0$ (approaching obstacle), tightens the constraint
- Together: they ensure $h(t)$ decays no faster than $e^{-\lambda t}$ where λ depends on α_1, α_2

Choosing α_1 and α_2:

The characteristic equation is $s^2 + \alpha_1 s + \alpha_2 = 0$. For $h(t) \geq 0$ to be guaranteed:

- Both roots must be **real and negative**: $\alpha_1^2 \geq 4\alpha_2$ and $\alpha_1, \alpha_2 > 0$
- Roots $= \frac{-\alpha_1 \pm \sqrt{\alpha_1^2 - 4\alpha_2}}{2}$
- Larger α_1, α_2 = more aggressive enforcement (intervenes earlier)

Variable drill-down:

Variable	What it is here
$\ddot{h}$	Second derivative of h — contains u (Eq. 7)
$\alpha_1 > 0$	**Damping coefficient** — controls how fast the approach rate $\dot{h}$ is damped
$\alpha_2 > 0$	**Stiffness coefficient** — controls how strongly the constraint reacts to h itself
$\alpha_1 \dot{h}$	Penalises rapid approach ($\dot{h} \ll 0$). Acts like velocity damping.
$\alpha_2 h$	Provides freedom proportional to safety margin. When h is large, the robot has more room.
≥ 0	The constraint — this is what goes into the QP as a linear inequality in u

Key insight: Substituting $\ddot{h} = L_f^2 h + L_g L_f h \cdot u$ gives a constraint of the form $Au \geq b$, which is a **linear inequality** in u. This is exactly what a QP can handle efficiently.

Equation (9) — ECBF constraint expanded for a double integrator

$$2(p-p_o)^\top u \geq -(2\|v\|^2 + 2\alpha_1(p-p_o)^\top v + \alpha_2(\|p-p_o\|^2 - r_{\text{safe}}^2))$$

Why it's here: This is Eq. 8 fully expanded for the specific case of a double integrator robot avoiding a circular obstacle. This is the **implementable** constraint — the thing you code up and pass to the QP solver.

What it is: A single linear inequality in u that guarantees the robot never collides with the obstacle.

Intuition — piece by piece:

Piece	Formula	Meaning
Left side	$2(p-p_o)^\top u$	Component of control acceleration in the direction away from obstacle. "How much are you pushing away?"
$2\|v\|^2$	Velocity magnitude term	Always positive — faster robot needs more room to brake
$2\alpha_1(p-p_o)^\top v$	Approach rate term	Negative when approaching — tightens the constraint. "You're heading toward it — brake harder."
$\alpha_2(\|p-p_o\|^2 - r_{\text{safe}}^2)$	Proximity term ($= \alpha_2 h$)	Positive when safe — loosens constraint. "You have margin — relax."

Variable drill-down:

Variable	What it is here
$(p-p_o)$	Vector from obstacle to robot ($n_p \times 1$)
$(p-p_o)^\top u$	Scalar: component of acceleration away from obstacle
$\|v\|^2$	Squared speed ($= v^\top v$)
$(p-p_o)^\top v$	Scalar: $= \dot{h}/2$ — rate of change of separation. Negative = approaching.
r_{safe}	$= R + r_o + \epsilon$ — minimum allowed centre-to-centre distance
$\|p-p_o\|^2 - r_{\text{safe}}^2$	$= h(p)$ — the barrier function value

Equation (10) — Nominal controller (PD)

$$u_{\text{nom}} = -k_1(p - p_d) - k_2 v$$

Why it's here: CBF is a **safety filter** — it modifies an existing controller. This equation defines that "existing controller." It's a simple PD (Proportional-Derivative) controller that drives the robot toward the goal. Without the CBF, this controller would ignore obstacles entirely.

What it is: A goal-seeking acceleration command with no obstacle awareness.

Intuition: Identical to DAF's first two terms:

- $-k_1(p - p_d)$: spring pulling toward goal (stronger when far away)
- $-k_2 v$: friction slowing the robot (prevents oscillation)

This is the controller the robot *wants* to use. The CBF's job is to modify it as little as possible to stay safe.

Variable drill-down:

Variable	What it is here
u_{nom}	**Nominal control** — the "desired" acceleration before safety filtering
k_1	Position gain — spring stiffness toward goal
k_2	Velocity gain — damping coefficient
$p - p_d$	Error vector from goal to robot
v	Current velocity

Comparison to DAF (Eq. 13): DAF's controller is $u = -k_1(p - p_d) - k_2 v - k_3 \gamma \eta \eta^\top v$. The first two terms are exactly u_{nom}. DAF bakes the avoidance into the third term; CBF handles it separately via the QP.

Equation (11) — The CBF-QP (Quadratic Program)

$$u^* = \underset{u \in \mathbb{R}^m}{\arg\min} \quad \|u - u_{\text{nom}}\|^2$$

$$\text{subject to:} \quad L_f^2 h(x) + L_g L_f h(x)\, u + \alpha_1 \dot{h} + \alpha_2 h \geq 0$$

Why it's here: This is the **engine** of the CBF approach — the thing that runs at every timestep. It takes the nominal controller's desire (u_{nom}) and finds the closest safe alternative (u^*). This is where the "minimal intervention" philosophy is implemented mathematically.

What it is: An optimisation problem: minimise deviation from the desired control, subject to the safety constraint.

Intuition: Imagine two people holding a steering wheel:

- The **nominal controller** wants to go straight toward the goal
- The **CBF** checks if that's safe
- If safe: $u^* = u_{\text{nom}}$ (CBF doesn't intervene at all)
- If unsafe: u^* is the closest control to u_{nom} that satisfies the safety constraint

Because the cost is quadratic ($\|\cdot\|^2$) and the constraint is linear in u, this QP has a **closed-form solution** for a single constraint:

$$u^* = \begin{cases} u_{\text{nom}}, & \text{if constraint satisfied by } u_{\text{nom}} \\ u_{\text{nom}} + \frac{\text{(violation)}}{L_g L_f h \cdot (L_g L_f h)^\top} (L_g L_f h)^\top, & \text{otherwise} \end{cases}$$

In words: project u_{nom} onto the feasible half-space defined by the constraint.

Variable drill-down:

Variable	What it is here
u^*	The **optimal safe control** — what the robot actually executes
$\arg\min$	"The value of u that minimises..."
$\|u - u_{\text{nom}}\|^2$	Squared deviation — the QP minimises this. Equal weight on all control dimensions.
"subject to"	The constraint that must be satisfied — the ECBF condition from Eq. 8
$L_f^2 h + L_g L_f h \cdot u + \alpha_1 \dot{h} + \alpha_2 h$	The left side of the safety constraint — linear in u

Key insight: The QP is solved at every timestep (e.g., every 1ms). For a single obstacle (one constraint, m variables), the solution is a simple projection — no iterative solver needed. For multiple obstacles, a general QP solver (like `scipy.optimize.minimize` or OSQP) is used.

Equation (12) — Forward invariance (safety guarantee)

$$h(x(0)) \geq 0 \implies h(x(t)) \geq 0 \quad \forall t \geq 0$$

Why it's here: This is the **main theorem** — the payoff of the entire CBF framework. If the system starts safe and the control always satisfies the CBF condition (Eq. 5 or 8), then the system stays safe forever. No collisions, guaranteed.

What it is: The forward invariance property of the safe set $\mathcal{C}$.

Intuition: Once inside the safe set, you can never leave. The CBF condition acts like a one-way door — the constraint at the boundary ($h = 0$) requires $\dot{h} \geq 0$, meaning the state can only move inward or stay on the boundary.

Variable drill-down:

Variable	What it is here
$h(x(0)) \geq 0$	The system starts in the safe set
$h(x(t)) \geq 0 \; \forall t$	The system stays in the safe set for all future time
$\implies$	Logical implication — if the left is true, the right is guaranteed

Proof sketch (Nagumo's theorem): The CBF condition ensures $\dot{h} \geq -\alpha(h)$ always. Consider what happens at the boundary ($h = 0$): $\dot{h} \geq -\alpha(0) = 0$, so h can't decrease — the state can't cross the boundary. By induction on the flow, $h(x(t)) \geq 0$ for all t.

Comparison to DAF: DAF proves safety using a Lyapunov function $\mathcal{L}$ that always decreases (Eq. 20) combined with a barrier term Φ that blows up at $d = 0$. CBF proves safety using Nagumo's theorem — a more direct "the set is invariant" argument. Both are valid but conceptually different.

Equation (13) — Feasibility condition

$$L_g L_f h(x) \neq 0 \quad \forall x \in \partial\mathcal{C}$$

Why it's here: The QP (Eq. 11) can only guarantee safety if the constraint is **feasible** — there must exist some u that satisfies it. If $L_g L_f h = 0$ at a point on the boundary, control has no effect on $\ddot{h}$ there, and the QP may be infeasible.

What it is: A regularity condition ensuring the control can always influence safety.

Intuition: If $L_g L_f h = 0$, the control input u has no effect on how h changes — it's like having a steering wheel that does nothing at the exact moment you need to turn. For obstacle avoidance with $h = \|p - p_o\|^2 - r^2$, we have $L_g L_f h = 2(p - p_o)^\top$, which is zero only when $p = p_o$ (robot is inside the obstacle). Since we start safe, this never happens.

Variable drill-down:

Variable	What it is here
$L_g L_f h(x)$	The "control effectiveness" for safety — how much u can influence $\ddot{h}$
$\neq 0$	Must be nonzero — otherwise the QP has no way to enforce the constraint

Variable	What it is here
$\partial\mathcal{C}$	The boundary — this is where feasibility matters most (the constraint is tightest there)

Equation (14) — Multiple obstacle extension

$$u^* = \arg\min_{u \in \mathbb{R}^m} \quad \|u - u_{\text{nom}}\|^2$$

$$\text{s.t.} \quad L_f^2 h_i + L_g L_f h_i \cdot u + \alpha_1 \dot{h}_i + \alpha_2 h_i \geq 0, \quad i = 1, \ldots, N_{obs}$$

Why it's here: Real environments have multiple obstacles. CBF handles this elegantly — just add one constraint per obstacle to the QP. This is a major advantage over DAF and APF, which only track the nearest obstacle and must switch between them.

What it is: The QP with one ECBF constraint per obstacle.

Intuition: Each obstacle gets its own "safety vote." The QP finds the control that:

1. Stays as close as possible to what the nominal controller wants
2. Satisfies **all** safety constraints simultaneously

If constraints conflict (e.g., two obstacles on opposite sides), the QP finds the best compromise. If no feasible u exists (completely boxed in), the QP is infeasible — this corresponds to a physically impossible scenario.

Variable drill-down:

Variable	What it is here
h_i	Barrier function for obstacle i: $h_i = \|p - p_{o,i}\|^2 - r_{\text{safe},i}^2$
N_{obs}	Total number of obstacles
"s.t."	"Subject to" — all constraints must hold simultaneously
$i = 1, \ldots, N_{obs}$	One constraint per obstacle

Computational note: With N_{obs} obstacles, the QP has m variables and N_{obs} inequality constraints. For a 2D robot with 10 obstacles, that's 2 variables and 10 constraints — trivially fast to solve.

Equation (15) — Gradient of the barrier function

$$\nabla h(p) = 2(p - p_o)$$

Why it's here: To implement the QP (Eq. 11), we need the gradient of h, which feeds into the Lie derivatives $L_f h$ and $L_g L_f h$. This equation computes it for the circular obstacle barrier.

What it is: The gradient of $h(p) = \|p - p_o\|^2 - r_{\text{safe}}^2$.

Intuition: The gradient points from the obstacle toward the robot — the direction of increasing safety. Its magnitude is $2\|p - p_o\|$ — larger when farther away.

Variable drill-down:

Variable	What it is here
$\nabla h(p)$	$n_p \times 1$ vector: direction of steepest increase in h
$2(p - p_o)$	Points from obstacle centre to robot, magnitude $2\|p - p_o\|$

Connection to DAF's η: The unit normal $\eta(p) = (p - p_o)/\|p - p_o\|$ in DAF is the normalised version of $\nabla h / \|\nabla h\|$. Both point away from the obstacle.

Equation (16) — Lie derivatives for obstacle avoidance

$$L_f h = 2(p - p_o)^\top v, \qquad L_g L_f h = 2(p - p_o)^\top$$

and

$$L_f^2 h = 2\|v\|^2$$

Why it's here: These are the concrete Lie derivatives needed to build the QP constraint (Eq. 9). They are computed by differentiating h along the system dynamics.

What they are:

- $L_f h = \dot{h}$: rate of change of the barrier function. Depends on velocity — negative means approaching.
- $L_f^2 h$: the drift contribution to $\ddot{h}$. Equals $2\|v\|^2$ — always non-negative, meaning speed alone makes h "want" to increase (centrifugal-like effect).
- $L_g L_f h$: the control effectiveness. The row vector $2(p - p_o)^\top$ shows that control in the direction away from the obstacle increases $\ddot{h}$.

Variable drill-down:

Variable	What it is here
$L_f h = 2(p - p_o)^\top v$	Dot product of separation direction and velocity. Negative = approaching obstacle. Equivalent to $2\|p - p_o\| \cdot \dot{d}$ where $\dot{d}$ is DAF's approach rate.
$L_f^2 h = 2\|v\|^2$	Always ≥ 0. Faster robots naturally cause $\ddot{h}$ to be more positive. Think of it as the centrifugal effect — speed creates an "outward push" in barrier space.
$L_g L_f h = 2(p - p_o)^\top$	A $1 \times m$ row vector. When multiplied by u: $2(p - p_o)^\top u$ = the acceleration component away from the obstacle, scaled by distance.

Equation (17) — Complete QP for a circular obstacle (implementable)

$$u^* = \arg\min_u \quad \|u - u_{\text{nom}}\|^2$$

$$\text{s.t.} \quad 2(p - p_o)^\top u + 2\|v\|^2 + 2\alpha_1 (p - p_o)^\top v + \alpha_2 h(p) \geq 0$$

Why it's here: This is the fully expanded, implementable QP for a single circular obstacle. This is what you code up. Compare with DAF's Eq. 13 — a closed-form formula vs. an optimisation problem.

What it is: The final CBF-QP combining all the pieces.

Implementation mapping:

What to compute	Formula	Source
u_{nom}	$-k_1(p - p_d) - k_2 v$	Eq. 10 (PD controller)
$h(p)$	$\|p - p_o\|^2 - r_{\text{safe}}^2$	Eq. 3
$\dot{h}$	$2(p - p_o)^\top v$	Eq. 16
Constraint LHS	$2(p - p_o)^\top u + 2\|v\|^2 + 2\alpha_1 (p - p_o)^\top v + \alpha_2 h$	Eqs. 7-9 combined
u^*	Solve QP (or closed-form projection for single constraint)	Eq. 11

Closed-form solution (single obstacle, derived by KKT conditions):

$$u^* = \begin{cases} u_{\text{nom}}, & \text{if } Au_{\text{nom}} + b \geq 0 \\ u_{\text{nom}} + \frac{-(Au_{\text{nom}} + b)}{AA^\top} A^\top, & \text{otherwise} \end{cases}$$

where $A = 2(p - p_o)^\top$ and $b = 2\|v\|^2 + 2\alpha_1 (p - p_o)^\top v + \alpha_2 h$.

Equation (18) — Structural comparison with DAF

DAF (Eq. 13):

$$u_{\text{DAF}} = \underbrace{-k_1(p - p_d)}_{\text{goal}} - \underbrace{k_2 v}_{\text{damping}} - \underbrace{k_3 \gamma(d)\, \eta\, \eta^\top v}_{\text{obstacle braking}}$$

CBF (Eq. 17):

$$u_{\text{CBF}} = \underbrace{u_{\text{nom}}}_{\text{goal + damping}} + \underbrace{\lambda \cdot (p - p_o)}_{\text{safety correction}}$$

where $\lambda \geq 0$ is the Lagrange multiplier from the QP (zero when safe, positive when constraint is active).

Why it's here: This side-by-side makes the philosophical difference precise.

Comparison table:

Aspect	DAF	CBF
Obstacle term	$-k_3 \gamma \eta (\eta^\top v)$ — always computed, proportional to approach speed and proximity	$\lambda(p - p_o)$ — only nonzero when constraint is active
When does it activate?	Whenever $d < \epsilon_2$ (enters braking zone)	Only when u_{nom} would violate the safety constraint
Direction	Along η (obstacle normal)	Along $p - p_o$ (away from obstacle)
Depends on velocity?	Yes — $\eta^\top v$ is the approach speed	Indirectly — through the constraint evaluation
Magnitude	Continuous, scales with $\gamma(d) = 1/d$ near wall	Discontinuous in its activation (on/off), but smooth when active
Tuning knobs	$k_3, \epsilon_1, \epsilon_2$	α_1, α_2
Can get stuck on flat walls?	Yes (Eq. 14-15 in DAF paper)	Yes, but differently — the nominal controller determines stuck behavior

Equation (19) — Structural comparison with APF

APF (DAF paper Eq. 18):

$$u_{\text{APF}} = -k_a(p - p_d) - k_v v + k_r F_r(p)$$

CBF (Eq. 17):

$$u_{\text{CBF}} = u_{\text{nom}} + \lambda \cdot (p - p_o)$$

Key difference: APF's repulsive force $F_r(p)$ depends only on position — it pushes even when moving away. CBF's correction $\lambda(p - p_o)$ is zero when the constraint isn't active — if u_{nom} is already safe, CBF doesn't intervene at all. This means CBF produces smoother motion when obstacles are nearby but not threatening.

Aspect	APF	CBF
Always pushes?	Yes — $F_r \neq 0$ whenever $d < \epsilon_2$	No — $\lambda = 0$ when u_{nom} is safe
Local minima	Common (repulsive + attractive forces cancel)	Possible but mitigated by QP feasibility
Jerkiness	High (sudden force changes)	Low (minimal modification to nominal)
Formal safety proof	No	Yes (forward invariance)

Complete Variable Reference Table

Symbol	Type	Defined	Meaning
n	scalar	given	State space dimension
m	scalar	given	Control input dimension
n_p	scalar	given	Position space dimension (2 or 3)
x	$\mathbb{R}^n$	state	Full state $[p^\top, v^\top]^\top$
p	$\mathbb{R}^{n_p}$	state	Robot centre position
v	$\mathbb{R}^{n_p}$	state	Robot velocity
u	$\mathbb{R}^m$	design	Control acceleration
u_{nom}	$\mathbb{R}^m$	Eq. 10	Nominal (goal-seeking) control

Symbol	Type	Defined	Meaning
u^*	$\mathbb{R}^m$	Eq. 11	Optimal safe control (QP output)
p_d	$\mathbb{R}^{n_p}$	given	Goal position
p_o	$\mathbb{R}^{n_p}$	given	Obstacle centre position
R	$\mathbb{R}_{>0}$	given	Robot radius
r_o	$\mathbb{R}_{>0}$	given	Obstacle radius
ϵ	$\mathbb{R}_{>0}$	design	Safety margin
r_{safe}	$\mathbb{R}_{>0}$	derived	$R + r_o + \epsilon$ — minimum centre-to-centre distance
$f(x)$	$\mathbb{R}^n$	Eq. 1	Drift dynamics
$g(x)$	$\mathbb{R}^{n\times m}$	Eq. 1	Control input matrix
$h(x)$	$\mathbb{R}$	Eq. 3	Barrier function
$\dot{h}$	$\mathbb{R}$	Eq. 7	Time derivative of barrier function
$\ddot{h}$	$\mathbb{R}$	Eq. 7	Second time derivative of barrier function
∇h	$\mathbb{R}^n$	Eq. 15	Gradient of h
$L_f h$	$\mathbb{R}$	Eq. 16	Lie derivative of h along f
$L_g h$	$\mathbb{R}^{1\times m}$	Eq. 5	Lie derivative of h along g
$L_f^2 h$	$\mathbb{R}$	Eq. 16	Second Lie derivative of h along f
$L_g L_f h$	$\mathbb{R}^{1\times m}$	Eq. 16	Mixed Lie derivative
$\alpha(\cdot)$	function	Eq. 4	Class $\mathcal{K}_\infty$ function
α_0	$\mathbb{R}_{>0}$	design	CBF enforcement gain (linear α)
α_1	$\mathbb{R}_{>0}$	Eq. 8	ECBF damping parameter
α_2	$\mathbb{R}_{>0}$	Eq. 8	ECBF stiffness parameter
k_1	$\mathbb{R}_{>0}$	Eq. 10	Nominal controller position gain
k_2	$\mathbb{R}_{>0}$	Eq. 10	Nominal controller damping gain

Symbol	Type	Defined	Meaning
λ	$\mathbb{R}_{\geq 0}$	Eq. 18	QP Lagrange multiplier
$\mathcal{C}$	set	Eq. 2	Safe set $\{x : h(x) \geq 0\}$
N_{obs}	$\mathbb{N}$	Eq. 14	Number of obstacles
$d(p)$	$\mathbb{R}$	derived	$\lvert p - p_o \rvert - r_{\text{safe}}$ — surface-to-surface distance
$\eta(p)$	unit vector	derived	$(p - p_o)/\lvert p - p_o \rvert$ — direction from obstacle to robot

---

Summary: Why CBF is Useful

1. **Modular** — Design your goal-seeking controller however you want, then add CBF as a safety layer. Swap controllers without changing the safety logic.
2. **Minimal intervention** — The QP modifies the nominal control only when necessary, producing smoother paths than APF or DAF when obstacles aren't threatening.
3. **Formally safe** — Forward invariance is proven mathematically. If the QP is feasible, safety is guaranteed.
4. **Multi-constraint** — Multiple obstacles, velocity limits, actuator bounds — just add more constraints to the QP. No redesign needed.
5. **Works in any dimension** — The theory is dimension-agnostic.
6. **Transparent** — You can inspect λ (the Lagrange multiplier) to see exactly when and how much the CBF intervenes.

Limitations

1. **QP solve per timestep** — More expensive than DAF/APF's closed-form controllers (though typically negligible for small obstacle counts).
2. **Feasibility not guaranteed** — If obstacles completely surround the robot, no safe u exists. DAF's $1/d$ barrier handles this more gracefully (it just brakes harder).
3. **Conservative with high α** — Aggressive enforcement can make the robot avoid obstacles too early. Tuning α_1, α_2 requires care.
4. **Nominal controller still matters** — CBF filters for safety but inherits the nominal controller's stuck points and local minima. If the PD controller drives into a corner, CBF prevents collision but doesn't help escape.

Summary: The Logical Chain

$$\underbrace{\text{Model system}}_{\text{Eq. 1}} \longrightarrow \underbrace{\text{Define safety}}_{\text{Eqs. 2-3}} \longrightarrow \underbrace{\text{CBF condition}}_{\text{Eqs. 4-5}} \longrightarrow \underbrace{\text{Handle rel. degree 2}}_{\text{Eqs. 6-9}} \longrightarrow \underbrace{\text{Nominal c}}_{\text{Eqs. 1}}$$

CBF — Simulation Ideas for Understanding

Ordered from simple to advanced.

---

1. Nominal PD Controller (No Safety)

- Use only: $u = -k_1(p - p_d) - k_2 v$
- Show the robot driving straight through obstacles
- Demonstrates why a safety filter is needed

2. Single Circular Obstacle with CBF-QP

- Implement the full QP (Eq. 17)
- Place goal on opposite side of obstacle
- Observe: robot curves around obstacle with minimal deviation from straight-line path
- Compare with DAF and APF trajectories

3. Visualise the Barrier Function $h(p)$

- Plot $h(p)$ as a 2D heatmap or contour plot
- Show the zero-level set ($h = 0$) as the safety boundary
- Overlay the robot trajectory on the contour plot

4. CBF vs DAF vs APF Side-by-Side (Single Obstacle)

- Identical initial conditions and goal
- Overlay all three trajectories
- Plot $\|u\|$, $\|v\|$, $d(t)$ over time
- Key observations: CBF intervenes later but more precisely; DAF brakes smoothly; APF jerks

5. QP Activation Visualization

- Plot the Lagrange multiplier $\lambda(t)$ over time
- $\lambda = 0$: CBF not intervening (nominal control is safe)

- $\lambda > 0$: CBF is actively modifying the control
- Shows the "minimal intervention" principle in action

6. Effect of α_1, α_2 on Conservatism

- Sweep α_1 and α_2
- Small values: robot gets very close to obstacles before CBF activates
- Large values: robot keeps a wide berth (conservative)
- Plot minimum distance vs. α parameters

7. Multiple Obstacles

- 3-5 obstacles between start and goal
- Show how the QP handles multiple constraints simultaneously
- Compare with DAF's nearest-obstacle-only approach

8. Flat Wall (CBF vs DAF Stuck Behavior)

- Same scenario as the DAF wall simulation
- Robot heading straight at a flat wall with goal behind it
- Both methods get stuck — but for different reasons
- DAF: equilibrium from force cancellation; CBF: nominal controller has no escape plan

9. Narrow Corridor

- Two parallel walls with a gap
- Tests feasibility of the QP with opposing constraints
- DAF may oscillate; CBF should find the gap if feasible

10. Moving Obstacles

- Add time-varying $p_o(t)$
- Requires recomputing $h, \dot{h}$ accounting for obstacle motion
- CBF handles this naturally by updating constraints each timestep

---

Minimum Priority Set

Priority	Simulation	What it proves
1	Single circle, CBF-QP (#2)	Core QP implementation works

Priority	Simulation	What it proves
2	CBF vs DAF vs APF comparison (#4)	Why CBF is different
3	$\lambda(t)$ visualization (#5)	Minimal intervention principle
4	Effect of α parameters (#6)	How to tune CBF
5	Flat wall comparison (#8)	Both methods' stuck behavior